

DOCKER_CHALLENGE

TASKS:

Create a customized Docker image using a Dockerfile.

Push the image to Docker Hub.

Push the same image to Amazon ECR.

Provision one EC2 instance using Terraform and install Jenkins.

Create one Jenkins job to build and push the Docker image to Docker Hub.

1. Source: <https://github.com/betawins/Python-app.git>



Source Codes: <https://github.com/betawins/docker-tasks.git>

From the frontend source code, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

From the Java-based source code, write a Dockerfile, build, run, and push the image to the Docker registry.

From the Node.js-based source code, write a Dockerfile, build with tag v1, run, and push it to the Docker registry.

- Write a docker-compose file to set up WordPress with a MySQL database.

1. Create a customized Docker image using a Dockerfile.

Push the image to Docker Hub.

Push the same image to Amazon ECR.

1 Create a Customized Docker Image

Example project structure

myapp/

└─ app.py

└─ Dockerfile

app.py

```
from flask import Flask
```

```
app = Flask(__name__)
```

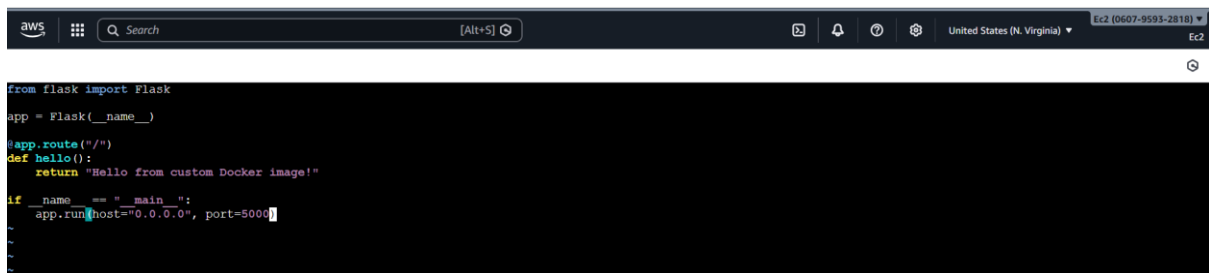
```
@app.route("/")

def hello():

    return "Hello from custom Docker image!"


if __name__ == "__main__":

    app.run(host="0.0.0.0", port=5000)
```

A screenshot of an AWS Management Console terminal window. The top bar shows the AWS logo, a search bar, and the region 'United States (N. Virginia)'. The instance ID 't2 (0607-9593-2818)' and instance type 't2' are visible. The terminal window displays the following Python code:

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def hello():
    return "Hello from custom Docker image!"
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dockerfile

Base image

FROM python:3.11-slim

Set working directory

WORKDIR /app

Install dependencies

RUN pip install flask

Copy application files

COPY app.py .

Expose port

EXPOSE 5000

Run application

CMD ["python", "app.py"]

```
# Base image
FROM python:3.11-slim

# Set working directory
WORKDIR /app

# Install dependencies
RUN pip install flask

# Copy application files
COPY app.py .

# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

Build the Docker image

docker build -t my-custom-app:1.0 .

Verify:

docker images

```
root@ip-172-31-27-214:~# mkdir myapp
root@ip-172-31-27-214:~# cd myapp
root@ip-172-31-27-214:~/myapp# vi app.py
root@ip-172-31-27-214:~/myapp# vi Dockerfile
root@ip-172-31-27-214:~/myapp# docker build -t my-custom-app:1.0 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/6 : FROM python:3.11-slim
3.11-slim: Pulling from library/python
0c8d55a45c0d: Already exists
64faa99400e1: Pulling fs layer
8cbc47ff628d: Pulling fs layer
d85099f0969e: Pulling fs layer
d85099f0969e: Download complete
64faa99400e1: Verifying Checksum
64faa99400e1: Download complete
8cbc47ff628d: Verifying Checksum
8cbc47ff628d: Download complete
64faa99400e1: Pull complete
8cbc47ff628d: Pull complete
d85099f0969e: Pull complete
Digest: sha256:0b23cfb7425d065008b778022a17b1551c82f8b4866ee5a7a200084b7e2eafbf
Status: Downloaded newer image for python:3.11-slim
--> 466c0182639b
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

→Image is created

```
root@ip-172-31-27-214:~/myapp# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
my-custom-app       1.0                f778898ea024       2 minutes ago      141MB
single-stage-app    <none>             678b99aca0f4d       2 days ago         254MB
single-stage-app    v1                 3ab55d0b1fca       2 days ago         554MB
arjundand10/alpine-tomcat v1                 3a15db1cad73       2 days ago         554MB
alpine-tomcat       v1                 ace9dab20d4d       2 days ago         300MB
060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat v1                 ace9dab20d4d       2 days ago         300MB
mynginximage        v1                 bf43af6d7194       2 days ago         161MB
eclipse-temurin     17-jdk-alpine     d3af761e11a8       3 days ago         335MB
nginx               latest             5cdef4ac3335       4 days ago         161MB
python              3.11-slim         466c0182639b       4 days ago         124MB
alpine              latest            a40c03cbb81c       12 days ago        8.44MB
maven               3.9.6-eclipse-temurin-17 b86805e902bf       2 years ago        481MB
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

2 Push Image to Docker Hub

Step 1: Login to Docker Hub

docker login

```
root@ip-172-31-27-214:~/myapp# docker login
Authenticating with existing credentials... [Username: arjumand10]
Info -> To login with a different account, run 'docker logout' followed by 'docker login'
Login Succeeded
root@ip-172-31-27-214:~/myapp#
```

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

Step 2: Tag the image

Docker Hub format:

<dockerhub-username>/<image-name>:<tag>

docker tag my-custom-app:1.0 arjumand10/my-custom-app:1.0

Step 3: Push the image

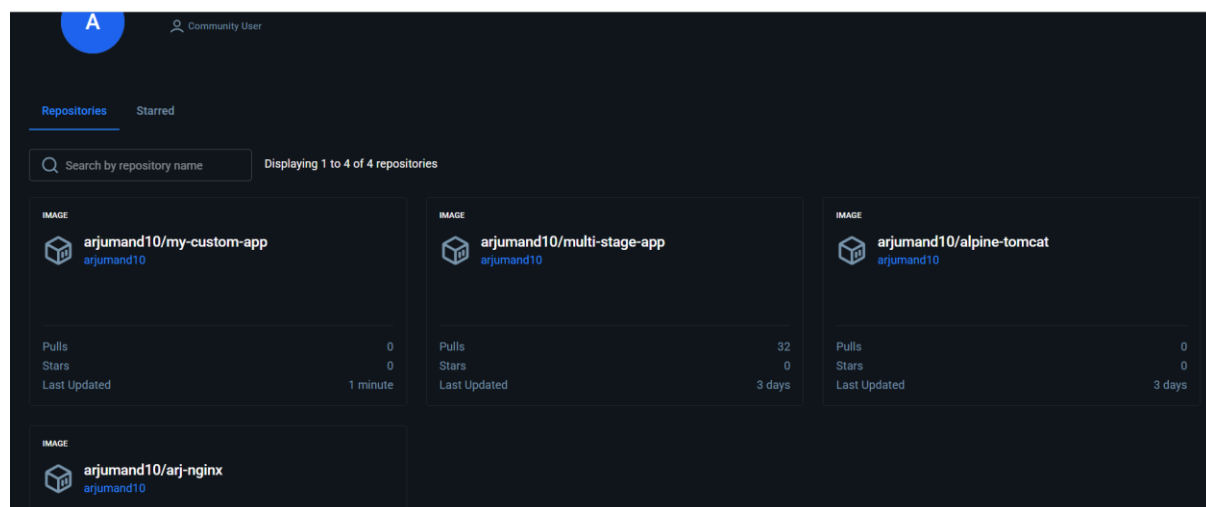
docker push arjumand10/my-custom-app:1.0

✅ Image is now available on **Docker Hub**

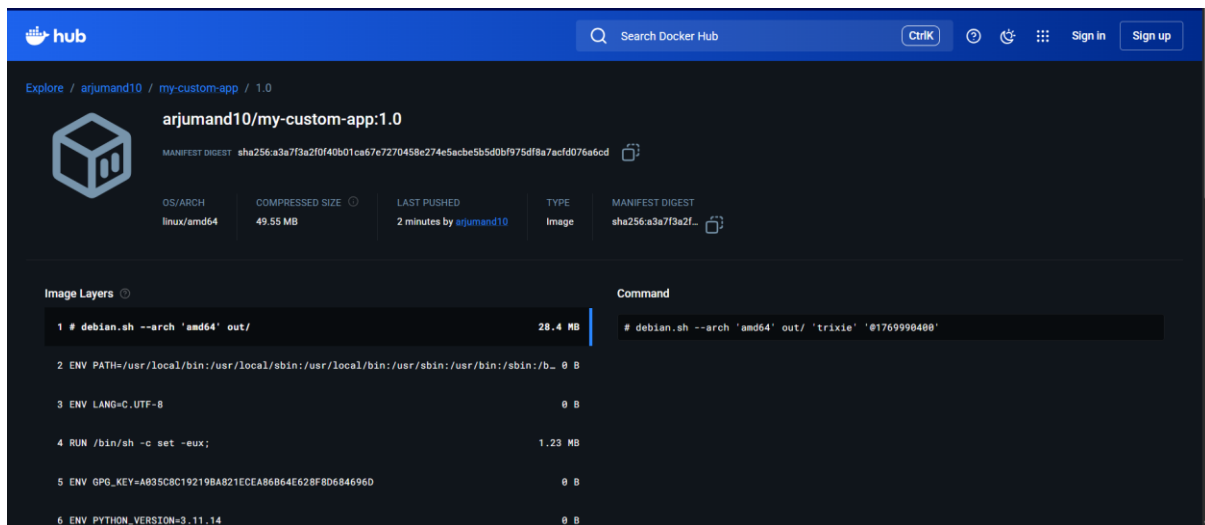
```
Login Succeeded
root@ip-172-31-27-214:~/myapp# docker tag my-custom-app:1.0 arjumand10/my-custom-app:1.0
root@ip-172-31-27-214:~/myapp# docker push arjumand10/my-custom-app:1.0
The push refers to repository [docker.io/arjumand10/my-custom-app]
c1ddabfe737e: Pushed
ab719961b6fe: Pushed
1a0ae4754504: Pushed
e69aea4cac7d: Mounted from library/python
40b88e8d19a2: Mounted from library/python
dfd9efb4ec4c: Mounted from library/python
a8ff6f8cbdfd: Mounted from library/python
1.0: digest: sha256:a3a7f3a2f0f40b01ca67e7270458e274e5acbe5b5d0bf975df8a7acfd076a6cd size: 1783
root@ip-172-31-27-214:~/myapp#
```

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

→ Verified with docker repo



→ Validation with tag



3 Push the Same Image to Amazon ECR

Prerequisites

- AWS account
- AWS CLI configured:

aws configure

- IAM permissions for ECR

```
root@ip-172-31-27-214:~/myapp# aws configure
AWS Access Key ID [*****ELEN]:
AWS Secret Access Key [*****5e0gl]:
Default region name [us-east-1]:
Default output format [json]:
root@ip-172-31-27-214:~/myapp#
```

Step 1: Create an ECR repository

aws ecr create-repository \

--repository-name my-custom-app \

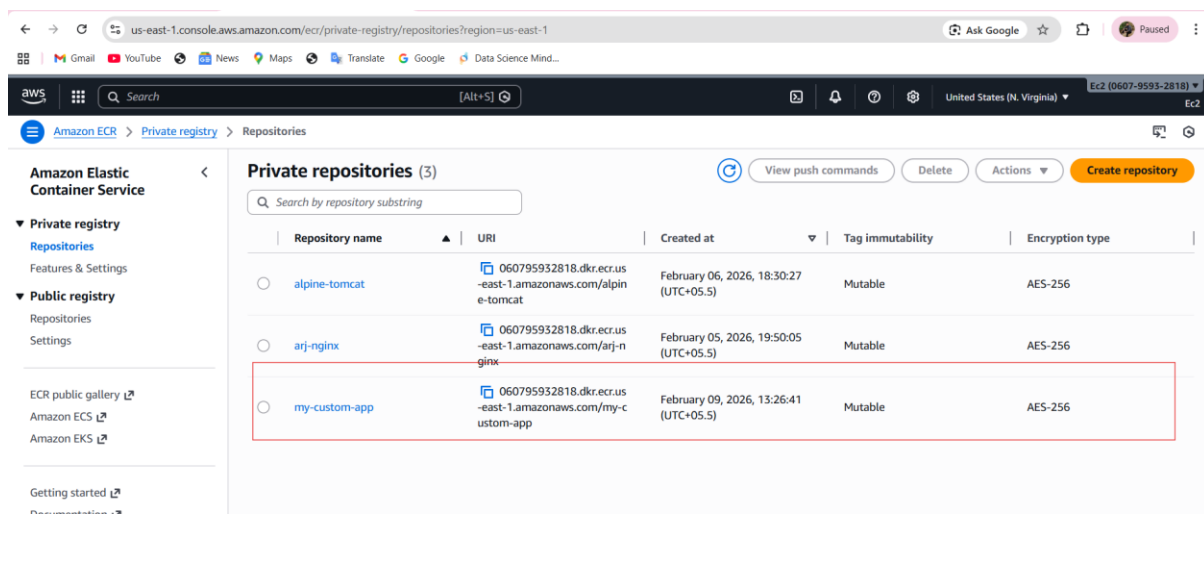
--region ap-south-1

```
root@ip-172-31-27-214:~/myapp# aws ecr create-repository \
--repository-name my-custom-app \
--region us-east-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:us-east-1:060795932818:repository/my-custom-app",
    "registryId": "060795932818",
    "repositoryName": "my-custom-app",
    "repositoryUri": "060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app",
    "createdAt": "2026-02-09T07:56:41.291000+00:00",
    "imageTagMutability": "Mutable",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

You'll get output like:

123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-app



Step 2: Login to Amazon ECR

```
aws ecr get-login-password --region us-east-1 \
| docker login --username AWS \
--password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com
```

```
root@ip-172-31-27-214:~/myapp# aws ecr get-login-password --region us-east-1 \
| docker login --username AWS \
--password-stdin 060795932818.dkr.ecr.us-east-1.amazonaws.com
WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/
Login Succeeded
root@ip-172-31-27-214:~/myapp#
```

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

Step 3: Tag the image for ECR

```
docker tag my-custom-app:1.0 \
```

123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-app:1.0

```
Login Succeeded
root@ip-172-31-27-214:~/myapp# docker tag my-custom-app:1.0 \
060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app:1.0
root@ip-172-31-27-214:~/myapp# docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app:1.0
The push refers to repository [060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app]
c1ddabfe737e: Pushed
ab719961b6fe: Pushed
1e0ae4754504: Pushed
b69aea4cac7d: Pushed
40b88e8d19a2: Pushed
dfd9efb4ec4c: Pushed
a8ff6f8cbdfd: Pushing [=====>] 49.54MB/78.62MB
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

Step 4: Push to ECR

docker push 123456789012.dkr.ecr.ap-south-1.amazonaws.com/my-custom-app:1.0

✓ Image is now stored in **Amazon ECR**

```
Login Succeeded
root@ip-172-31-27-214:~/myapp# docker tag my-custom-app:1.0 \
060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app:1.0
root@ip-172-31-27-214:~/myapp# docker push 060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app:1.0
The push refers to repository [060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app]
c1ddabfe737e: Pushed
ab719961b6fe: Pushed
1e0ae4754504: Pushed
b69aea4cac7d: Pushed
40b88e8d19a2: Pushed
dfd9efb4ec4c: Pushed
a8ff6f8cbdfd: Pushing [=====>] 49.54MB/78.62MB
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

→Validation:

Amazon ECR > Private registry > Repositories > Images

Amazon Elastic Container Service

Private registry

Repositories

my-custom-app

Features & Settings

Public registry

Repositories

Settings

ECR public gallery

my-custom-app

Summary Images Lifecycle policy Permissions Repository tags

Images (1) Info

Filter active images

☐	Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
☐	1.0	Image	February 09, 2025, 13:32:27 (UTC+05.5)	52.96	sha256:69bfa7a2...	-

✓ **Summary Flow**

Dockerfile



docker build



docker tag

└— docker push (Docker Hub)

└─ docker push (Amazon ECR)

2. Provision one EC2 instance using Terraform and install Jenkins.

Provision EC2 with Terraform + Install Jenkins

Create folder:

```
mkdir terraform-jenkins
cd terraform-jenkins
```

main.tf

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_security_group" "jenkins_sg" {
  name        = "jenkins-sg"
  description = "Allow SSH and Jenkins"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 8080
    to_port   = 8080
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "jenkins" {
  ami          = "ami-0b6c6ebed2801a5cb"
  instance_type = "t2.micro"
  key_name     = "your-keypair-name"

  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    sudo apt update -y
    sudo apt install openjdk-17-jdk -y

    sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
```



```
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]"
```

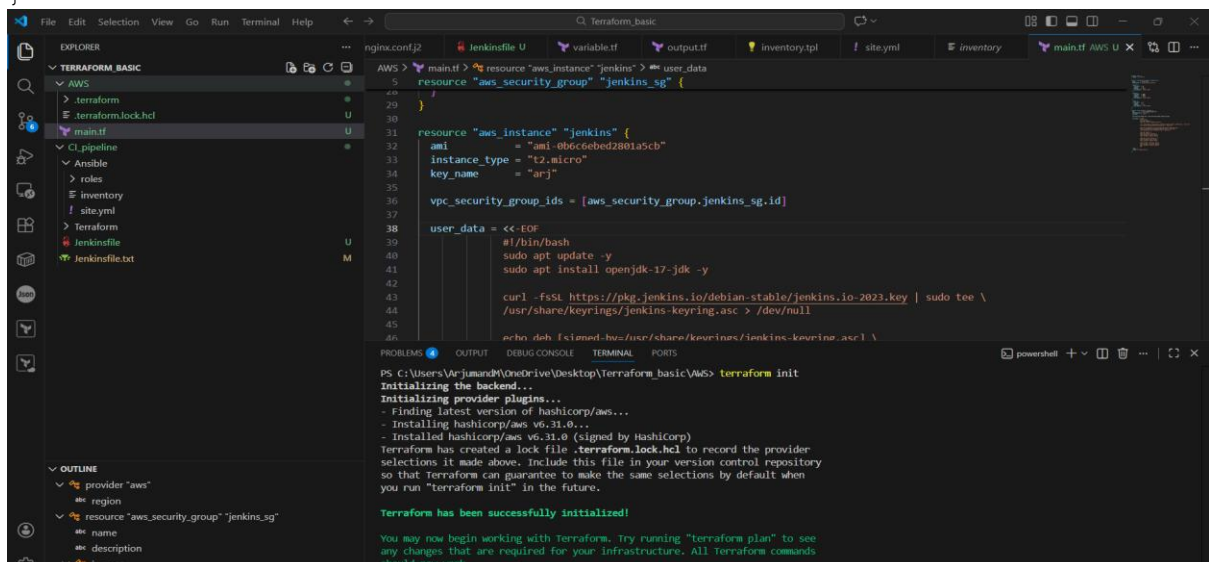
```
\
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt update
sudo apt install jenkins
```

```
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

```
sudo usermod -aG docker ubuntu
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
EOF
```

```
tags = {
  Name = "Jenkins-Server"
}
```



Run Terraform Commands

```
terraform init
```

```
PS C:\Users\Arjumand\OneDrive\Desktop\Terraform_basic\AWS> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.31.0...
- Installed hashicorp/aws v6.31.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
```

```
terraform validate
```

```
terraform plan
```

```
Commands will detect it and remind you to do so if necessary.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\AWS> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.jenkins will be created
+ resource "aws_instance" "jenkins" {
  + ami                  = "ami-0b6c6ebd2801a5cb"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + disable_api_stop      = (known after apply)
}
```

terraform apply -auto-approve

```
AWS > main.tf > resource "aws_instance" "jenkins" > user_data
5 resource "aws_security_group" "jenkins_sg" {
29 }
30
31 resource "aws_instance" "jenkins" {
32   ami = "ami-0b6c6ebd2801a5cb"
33   instance_type = "t2.micro"
34   key_name = "arj"
35
36   vpc_security_group_ids = [aws_security_group.jenkins_sg.id]
37
38   user_data = <<-EOF
39   #!/bin/bash
40   sudo apt update -y
41   sudo apt install openjdk-17-jdk -y
42
43   curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
44   /usr/share/keyrings/jenkins-keyring.asc > /dev/null
45
46   echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \

```

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_security_group.jenkins_sg: Creating...
aws_security_group.jenkins_sg: Creation complete after 6s [id=sg-00b17303418b745b8]
aws_instance.jenkins: Creating...
aws_instance.jenkins: Still creating... [00m10s elapsed]
aws_instance.jenkins: Creation complete after 16s [id=i-0f8b66749b1cbe12e]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic\AWS>

→Validation:

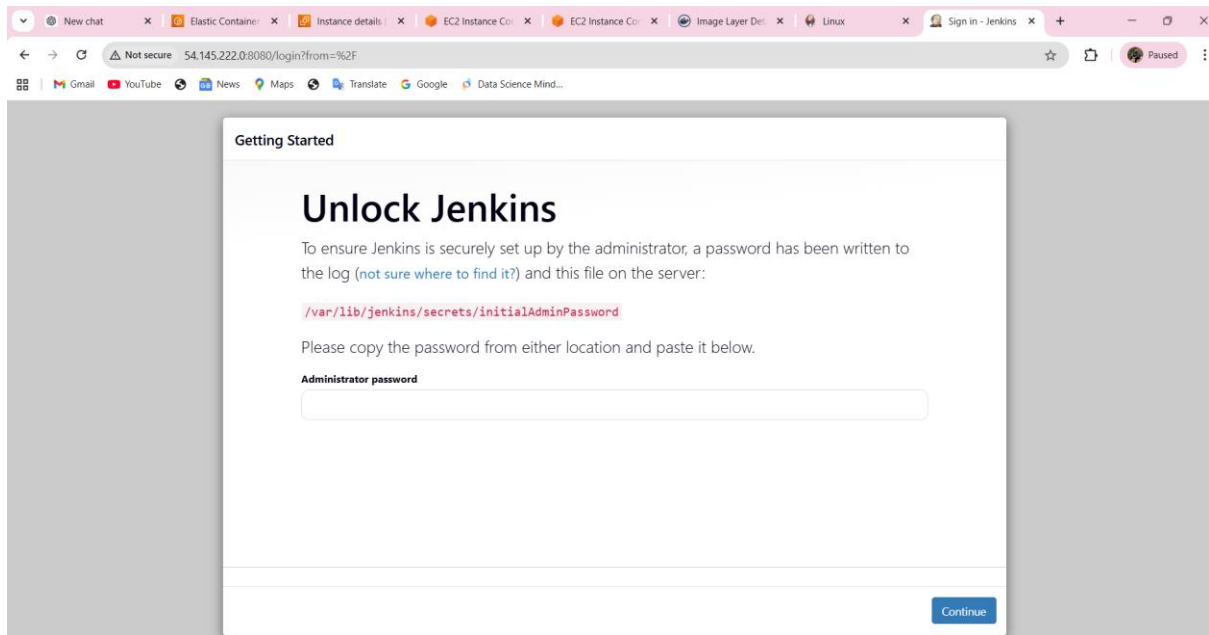
One Jenkins server ec2 has been created

☐	u21.local	i-02882c20ce32a5af6	⏸ Stopped	t2.micro	-	View alarms	us-east
☐	Docker_controller	i-028f1dc9ea91e8b89	🟢 Running	t2.large	🟢 2/2 checks passed	View alarms	us-east
☐	Jenkins-Server	i-0f8b66749b1cbe12e	🟢 Running	t2.micro	🕒 Initializing	View alarms	us-east
☐	Master_jenkin	i-04e9dd922f331d1cf	⏸ Stopped	t2.large	-	View alarms	us-east
☐	MyAnsibleInstance	i-005bfe901b6d46efa	⏸ Stopped	t2.micro	-	View alarms	us-east
☐	ansible-master	i-016f1e3b5d92425c4	⏸ Stopped	t2.micro	-	View alarms	us-east

Jenkins Access

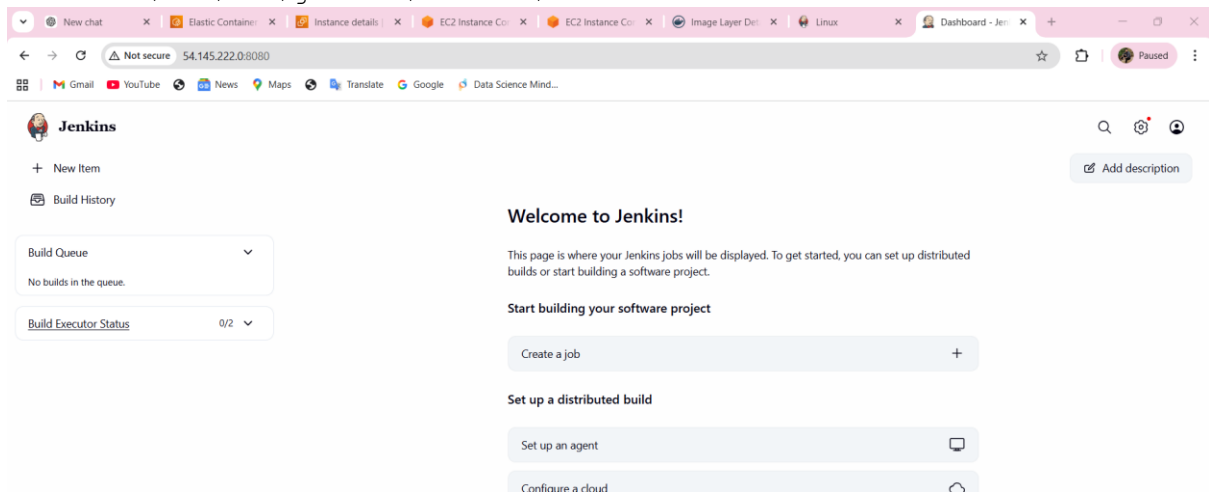
After Terraform completes, copy EC2 Public IP and open:

👉 `http://<EC2-PUBLIC-IP>:8080`



To get Jenkins password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```



Jenkins Pipeline Job (Build + Push DockerHub)

Jenkins Plugins Install

Go to:

Manage Jenkins → Plugins

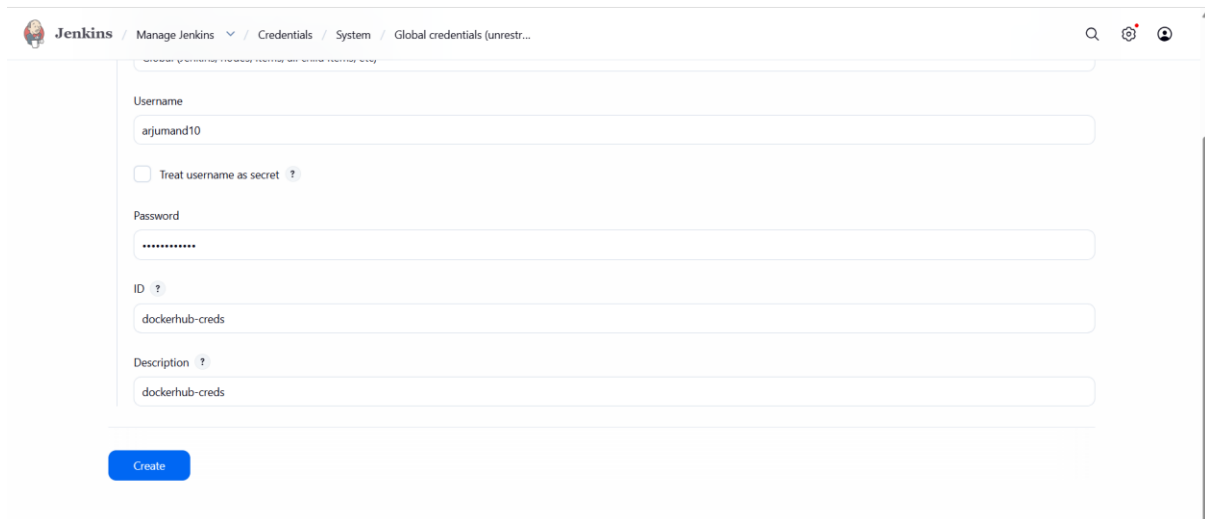
Install:

- ✓ Docker Pipeline
- ✓ Git Plugin
- ✓ Credentials Binding

Add DockerHub Credentials

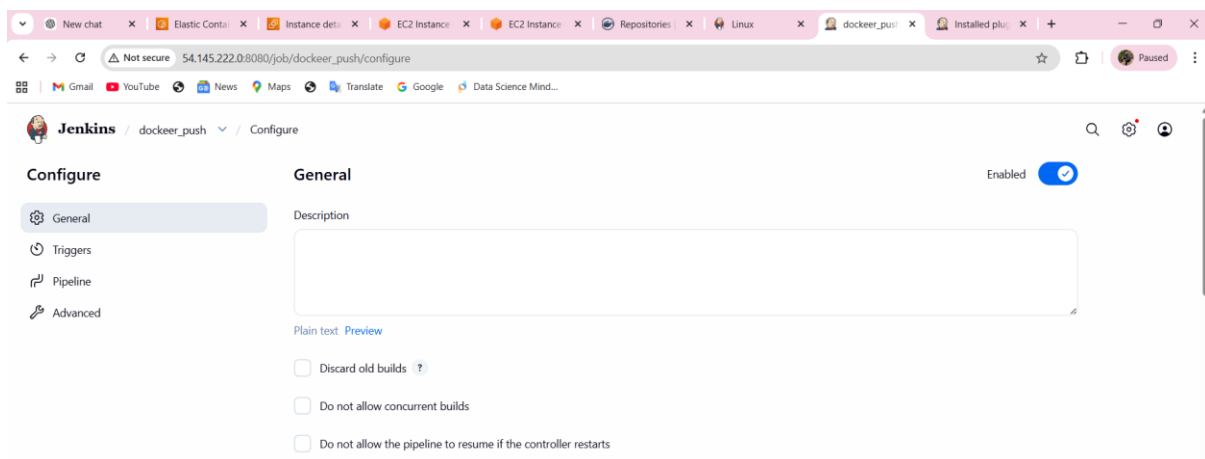
Manage Jenkins → Credentials → Global → Add Credentials

- Type: Username with password
- ID: dockerhub-creds
- Username: DockerHub username
- Password: DockerHub password



The screenshot shows the Jenkins 'Add Credentials' form. The breadcrumb trail at the top is 'Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestricted)'. The form contains the following fields: 'Username' with the value 'arjumand10', a checkbox for 'Treat username as secret' which is unchecked, 'Password' with masked characters, 'ID' with the value 'dockerhub-creds', and 'Description' with the value 'dockerhub-creds'. A blue 'Create' button is at the bottom left.

→ Create a job



The screenshot shows the Jenkins 'Configure' page for a Pipeline Job. The breadcrumb trail is 'Jenkins / docker_push / Configure'. On the left, the 'Configure' section has tabs for 'General', 'Triggers', 'Pipeline', and 'Advanced', with 'General' selected. The 'General' tab shows a 'Description' field, a 'Plain text' link, and three unchecked checkboxes: 'Discard old builds', 'Do not allow concurrent builds', and 'Do not allow the pipeline to resume if the controller restarts'. On the right, the job is 'Enabled' with a blue toggle switch.

Create Pipeline Job

Jenkins Dashboard → New Item → Pipeline Job

Name: Python-Docker-Push

Pipeline script:

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "python-app"
        IMAGE_TAG = "v1"
        DOCKERHUB_USER = "arjumand10"
    }

    stages {

        stage('Clone Repository') {
            steps {
                git branch: 'main', url: 'https://github.com/arjumand9794/Python-app.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
            }
        }

        stage('Tag Image') {
            steps {
                sh "docker tag ${IMAGE_NAME}:${IMAGE_TAG}
${DOCKERHUB_USER}/${IMAGE_NAME}:${IMAGE_TAG}"
            }
        }
    }
}
```

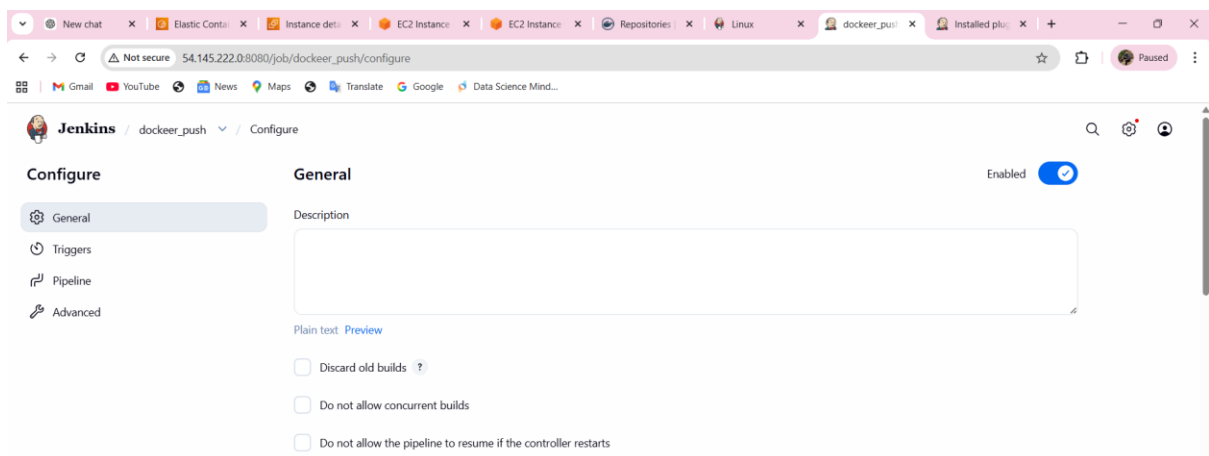
```

    }

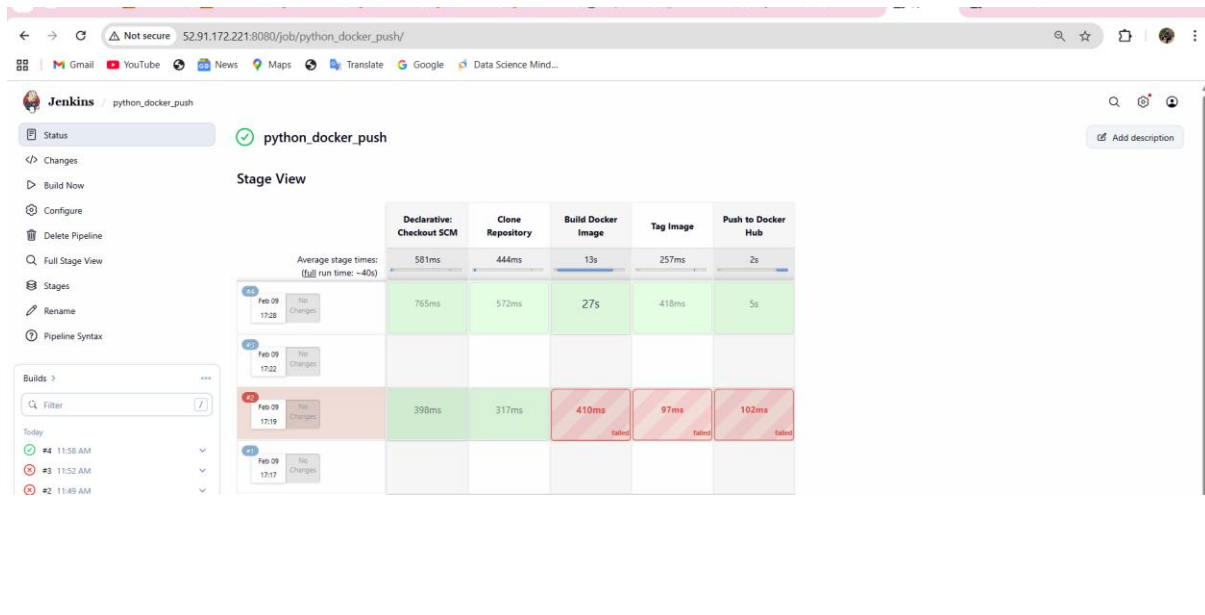
    stage('Push to Docker Hub') {
        steps {
            withCredentials([usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )]) {
                sh """
                echo \${DOCKER_PASS} | docker login -u \${DOCKER_USER} --password-stdin
                docker push \${DOCKERHUB_USER}/\${IMAGE_NAME}:\${IMAGE_TAG}
                """
            }
        }
    }
}

```

}Run Build Now 



→Validation:



3. Source Codes: <https://github.com/betawins/docker-tasks.git>

From the frontend source code, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

✓ STEP 10: Clone docker-tasks Repo

git clone https://github.com/betawins/docker-tasks.git

cd docker-tasks

Clone docker-tasks Repo

git clone https://github.com/betawins/docker-tasks.git

cd docker-tasks

```
root@ip-172-31-27-214:~# git clone https://github.com/arjumand9794/docker-tasks.git
Cloning into 'docker-tasks'...
remote: Enumerating objects: 146, done.
remote: Counting objects: 100% (146/146), done.
remote: Compressing objects: 100% (123/123), done.
remote: Total 146 (delta 12), reused 141 (delta 9), pack-reused 0 (from 0)
Receiving objects: 100% (146/146), 22.11 MiB | 33.49 MiB/s, done.
Resolving deltas: 100% (12/12), done.
root@ip-172-31-27-214:~# ls
alpine-tomcat  aws  awscli.v2.zip  compose-demo  docker-tasks  multi-stage-example  myapp  nginx-task  snap
root@ip-172-31-27-214:~# ls -la
.  .aws  .bashrc  .git  .profile  .viminfo  alpine-tomcat  awscli.v2.zip  docker-tasks  myapp  snap
..  .bash_history  .docker  .lsshst  .ssh  .wget-hsts  aws  compose-demo  multi-stage-example  nginx-task
root@ip-172-31-27-214:~# cd docker-tasks/
root@ip-172-31-27-214:~/docker-tasks# ls
Frontend_based_source  Java_based_source  NodeJs_based_source  README.md
root@ip-172-31-27-214:~/docker-tasks# cd Frontend_based_source/
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# ls
index.html  javascript.js  style.css  todayDeal.js
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source#
```

i-028f1dc9ea91e8b89 (Docker_controller)

PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

✓ STEP 11: Frontend Dockerfile (React)

Go to frontend folder (example):

```
cd frontend
```

```
Create Dockerfile
```

```
# Base image
```

```
FROM nginx:latest
```

```
# Maintainer name
```

```
LABEL maintainer="Arjumand Mohiuddin"
```

```
# Remove default nginx content
```

```
RUN rm -rf /usr/share/nginx/html/*
```

```
# Copy custom index.html
```

```
COPY index.html /usr/share/nginx/html/index.html
```

```
# Expose port 80
```

```
EXPOSE 80
```

```
# Start nginx
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
5#. Expose on port 80
```

```
EXPOSE 80
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```



```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build

docker build -t frontend-app:v1 .

```
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# vi Dockerfile
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# docker build -t frontend-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
              Install the buildx component to build images with BuildKit:
              https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  44.03kB
Step 1/6 : FROM nginx:latest
--> 5cdef4ac3335
Step 2/6 : LABEL maintainer="Arjumand Mohiuddin"
--> Running in 7063b5e799d9
--> Removed intermediate container 7063b5e799d9
--> 53efc4a9a1b1
Step 3/6 : RUN rm -rf /usr/share/nginx/html/*
--> Running in 52a9507a70aa
--> Removed intermediate container 52a9507a70aa
--> a15d9041f698
Step 4/6 : COPY index.html /usr/share/nginx/html/index.html
--> 7c1b42122dea
Step 5/6 : EXPOSE 80
--> Running in c900a64926e3
--> Removed intermediate container c900a64926e3
--> 3b9d4cdd7dd2
Step 6/6 : CMD ["nginx", "-g", "daemon off;"]

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104  PrivateIPs: 172.31.27.214
```

Run

docker run -itd --name frontend_container -p 8081:80 frontend-app:v1

Push DockerHub

docker tag frontend-app:v1 <dockerhub-username>/frontend-app:v1

```
Successfully tagged frontend-app:v1
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# docker run -itd --name frontend_container -p 8081:80 frontend-app:v1
1c7228dec884e404a82475b6cb4743ed87317d0cb963f3f27cc05a016c7b5c96
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
frontend-app        latest             773e2a52089c       5 minutes ago      161MB
060795932818.dkr.ecr.us-east-1.amazonaws.com/my-custom-app 1.0               f773898ca024       5 hours ago        141MB
arjumand10/my-custom-app 1.0               f773898ca024       5 hours ago        141MB
my-custom-app        latest             f773898ca024       5 hours ago        141MB
multi-stage-app      v1                678b9aca0f4d       3 days ago         354MB
<none>               <none>            3ab55d0b1fca       3 days ago         554MB
single-stage-app     v1                3a15db1cad73       3 days ago         554MB
060795932818.dkr.ecr.us-east-1.amazonaws.com/alpine-tomcat  v1                ace9dab20d4d       3 days ago         300MB
arjumand10/alpine-tomcat v1                ace9dab20d4d       3 days ago         300MB
alpine-tomcat        v1                bf43af6d7194       3 days ago         161MB
mynginximage         v1                d3af761e11a8       3 days ago         335MB
eclipse-temurin      17-jdk-alpine     5cdef4ac3335       4 days ago         161MB
nginx                 latest             466c0182639b       4 days ago         124MB
python                3.11-slim          a40c03cbb81c       12 days ago        8.44MB
alpine                latest             b86805e902bf       2 years ago        481MB
maven                 3.9.6-eclipse-temurin-17
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED             STATUS              PORTS                               NAMES
1c7228dec884   frontend-app:v1 /docker-entrypoint.sh    22 seconds ago     Up 21 seconds      0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  frontend_container

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104  PrivateIPs: 172.31.27.214
```

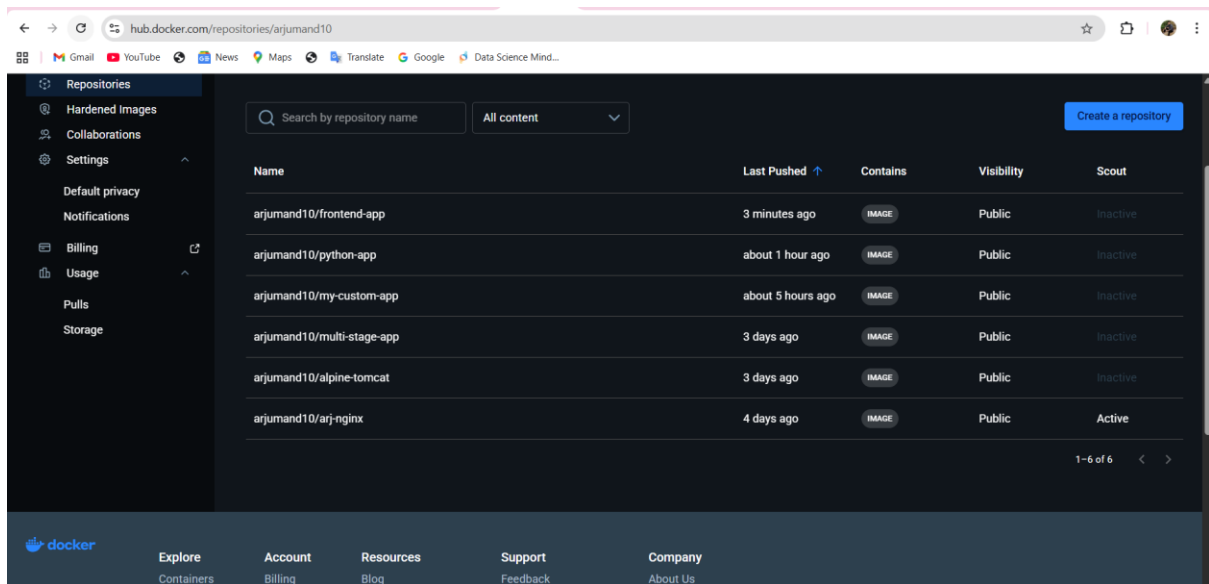
docker push <dockerhub-username>/frontend-app:v1

```
-bash: dockerhub-username: No such file or directory
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# docker tag frontend-app:v1 arjumand10/frontend-app:v1
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# docker tag arjumand10/frontend-app:v1
docker: 'docker tag' requires 2 arguments

Usage:  docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]

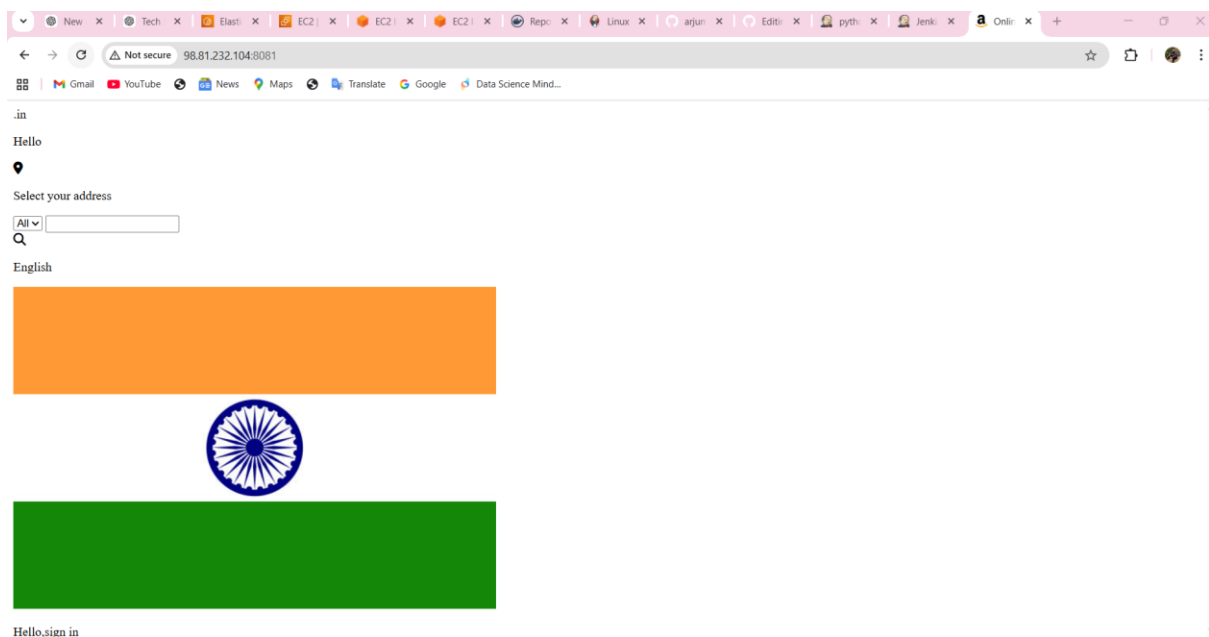
Run 'docker tag --help' for more information
root@ip-172-31-27-214:~/docker-tasks/Frontend_based_source# docker push arjumand10/frontend-app:v1
The push refers to repository [docker.io/arjumand10/frontend-app]
3da06f8f1448: Pushed
2693ba3e108e: Pushed
4952de04fe7e: Mounted from library/nginx
627c009aa115: Mounted from library/nginx
b9390f60b84f: Mounted from library/nginx
baed0e9a39b0: Mounted from library/nginx
18b09c39ca9f: Mounted from library/nginx
edcb98f6af68: Mounted from library/nginx
a8ff6f8cbdfd: Mounted from arjumand10/my-custom-app

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104  PrivateIPs: 172.31.27.214
```



→ Validation:

Public repo with port number is working on the web



4. From the Java-based source code, write a Dockerfile, build, run, and push the image to the Docker registry.

Java Dockerfile

Go to Java folder:

```
cd java-app
```

Create Dockerfile

FROM maven:3.9.6-eclipse-temurin-17 AS build

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

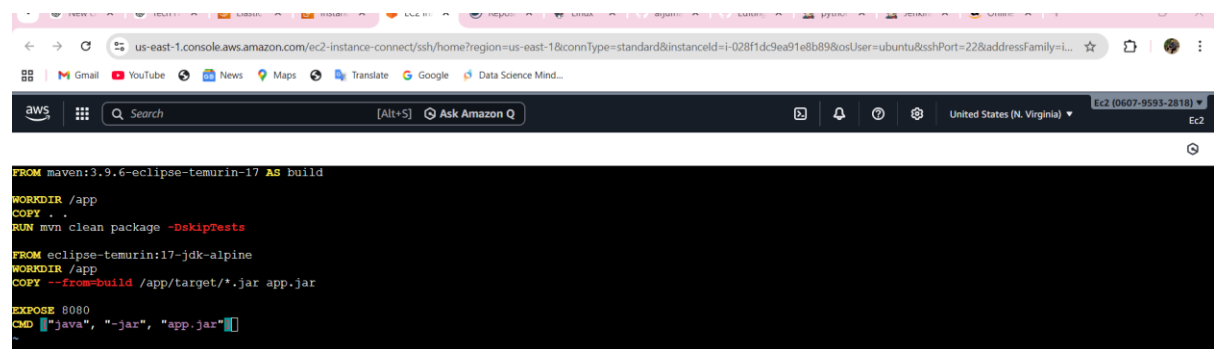
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]



The screenshot shows a terminal window within the AWS console. The terminal displays the following Dockerfile instructions:

```
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Build

docker build -t java-app:v1 .



The screenshot shows a terminal window with the output of the 'docker build' command. The output indicates a successful build of the 'java-app:v1' image. Key details include:

- Replacing main artifact with repackaged archive
- BUILD SUCCESS
- Total time: 16.396 s
- Finished at: 2026-02-09T13:25:03Z
- Removed intermediate container 03fec3253b1d
- Step 5/9: FROM eclipse-temurin:17-jdk-alpine
- Step 6/9: WORKDIR /app
- Step 7/9: COPY --from=build /app/target/*.jar app.jar
- Step 8/9: EXPOSE 8080
- Step 9/9: CMD ["java", "-jar", "app.jar"]
- Successfully built 6f28c6c18b65
- Successfully tagged java-app:v1

The terminal also shows the IP addresses for the Docker controller: PublicIPs: 98.81.232.104, PrivateIPs: 172.31.27.214.

Run

docker run -itd --name java_container -p 8082:8080 java-app:v1

Push DockerHub

```
docker tag java-app:v1 <dockerhub-username>/java-app:v1
```

```
docker push <dockerhub-username>/java-app:v1
```

```
----> Removed intermediate container d1956ccad183
----> 6f28c6c18b65
Successfully built 6f28c6c18b65
Successfully tagged java-app:v1
root@ip-172-31-27-214:~/docker-tasks/Java_based_source# docker run -itd --name java_container -p 8082:8080 java-app:v1
b742a4cba5457b1b8b636de76d4f6692357eal66fb29df56d257628d6d8439c
root@ip-172-31-27-214:~/docker-tasks/Java_based_source# docker tag java-app:v1 arjumand10/java-app:v1
root@ip-172-31-27-214:~/docker-tasks/Java_based_source# docker push arjumand10/java-app:v1
The push refers to repository [docker.io/arjumand10/java-app]
41666394ce70: Pushed
a371cb30e035: Pushed
f7409d4c66cc: Mounted from library/eclipse-temurin
08e98c779fb9: Mounted from library/eclipse-temurin
0a828e8088fe: Mounted from library/eclipse-temurin
344ae0b6479e: Mounted from library/eclipse-temurin
989e799e6349: Mounted from arjumand10/alpine-tomcat

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214
```

→Validation

```
989e799e6349: Mounted from arjumand10/alpine-tomcat
v1: digest: sha256:f6dd3b3152abf1e0e124d7f4cf4c7cdf9002e2c3c6b47baecf657fb2cb060bb1 size: 1786
root@ip-172-31-27-214:~/docker-tasks/Java_based_source# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
b742a4cba545   java-app:v1    "/_cacert_entrypoint..." About a minute ago Up About a minute    0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp    java_container
1c722d0ee894   frontend-app:v1 "/docker-entrypoint..." 18 minutes ago Up 18 minutes    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp        frontend_container
root@ip-172-31-27-214:~/docker-tasks/Java_based_source# docker image
Usage: docker image COMMAND

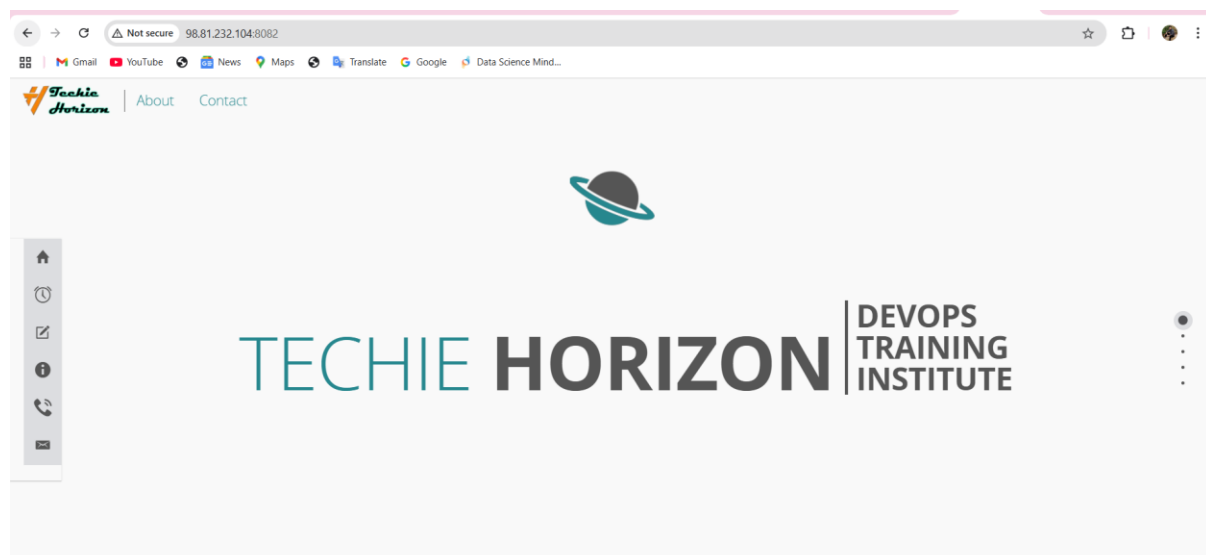
Manage images

Commands:
  build      Build an image from a Dockerfile
  history    Show the history of an image
  import     Import the contents from a tarball to create a filesystem image
  inspect    Display detailed information on one or more images
  load       Load an image from a tar archive or STDIN
  ls         List images
  prune      Remove unused images
  pull       Download an image from a registry
  push       Upload an image to a registry
  rm         Remove one or more images
  save       Save one or more images to a tar archive (streamed to STDOUT by default)
  tag        Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

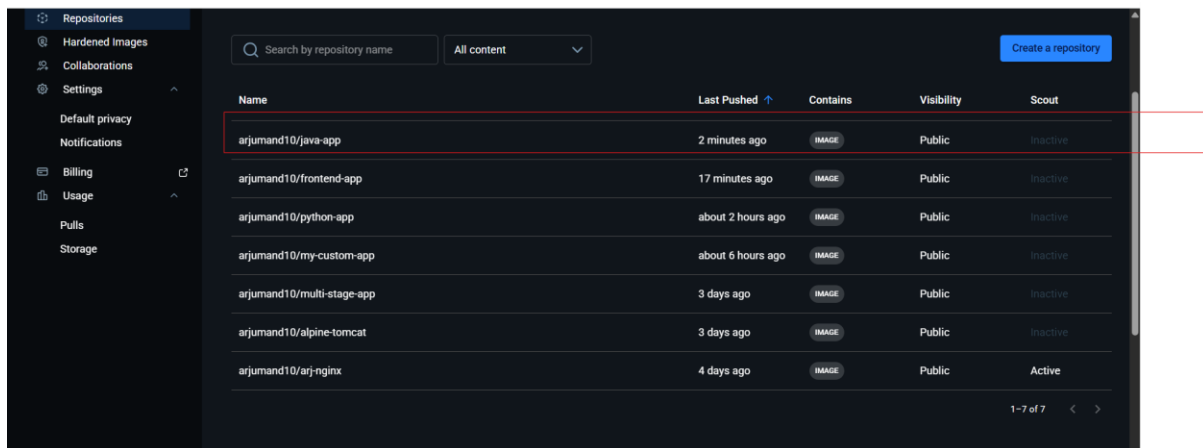
Run 'docker image COMMAND --help' for more information on a command.
root@ip-172-31-27-214:~/docker-tasks/Java_based_source#

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214
```

→validation on the web using port number 8082



→Validation in docker hub



5. From the Node.js-based source code, write a Dockerfile, build with tag v1, run, and push it to the Docker registry

✅ STEP 13: Node.js Dockerfile (Tag v1)

Go to node folder:

```
cd node-app
```

Create Dockerfile

```
FROM node:18-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["node", "index.js"]
```

```
FROM node:18-alpine
# create app directory
WORKDIR /app
# copy dependency files first (cache optimization)
COPY package*.json ./
# install dependencies
RUN npm install
# copy remaining source code
COPY . .
EXPOSE 3000

CMD ["npm", "start"]
```

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

Build Tag v1

docker build -t node-app:v1 .

```
root@ip-172-31-27-214:~/docker-tasks/NodeJs based_source# docker build -t node-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.17MB
Step 1/7 : FROM node:18-alpine
18-alpine: Pulling from library/node
f18232174bc9: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Pulling fs layer
25ff2da83641: Waiting
1e5a4c89cee5: Verifying Checksum
1e5a4c89cee5: Download complete
25ff2da83641: Verifying Checksum
25ff2da83641: Download complete
f18232174bc9: Download complete
f18232174bc9: Pull complete
dd71dde834b5: Verifying Checksum
dd71dde834b5: Download complete
```

Run

docker run -d -p 3000:3000 node1-app:v1

Push DockerHub

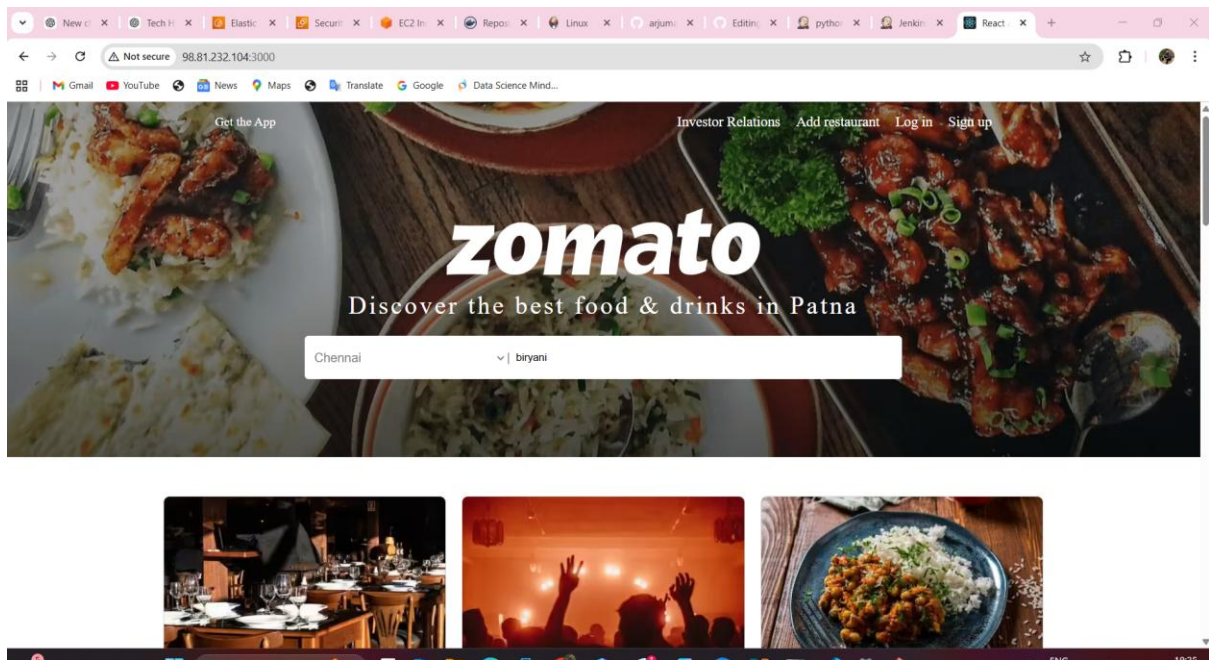
docker tag node1-app:v1 <dockerhub-username>/node-app:v1

docker push <dockerhub-username>/node1-app:v1

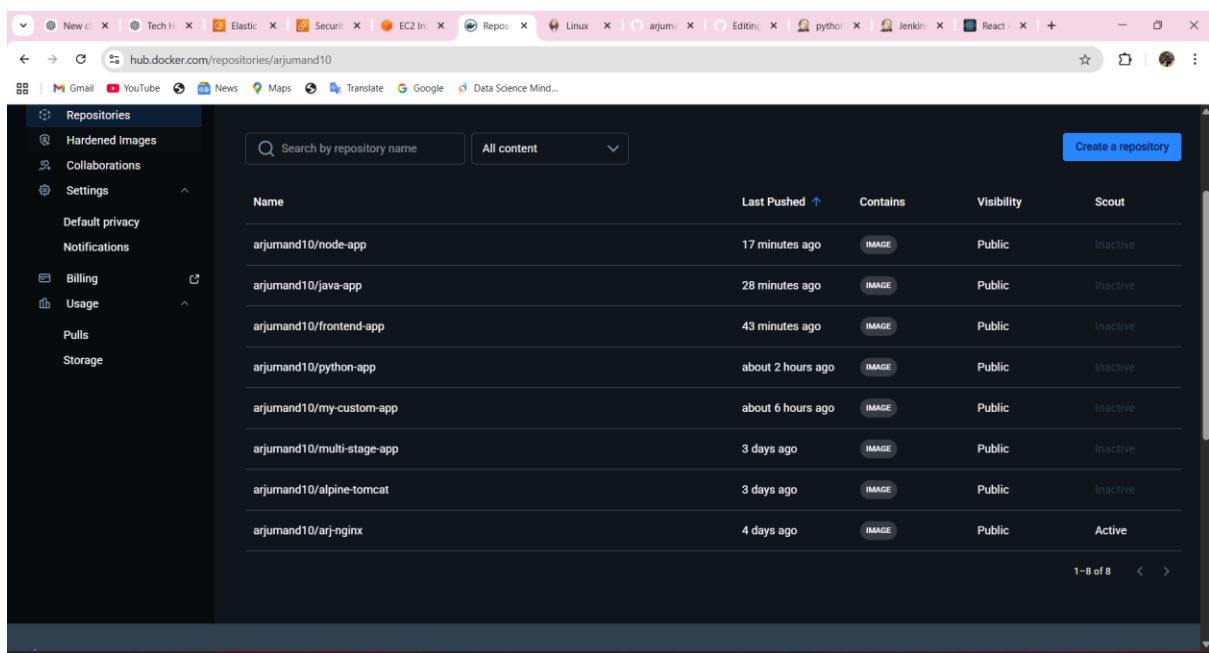
```
Step 6/7 : EXPOSE 3000
--> Running in 235ac2258282
--> Removed intermediate container 235ac2258282
--> be9d1cb3c59c
Step 7/7 : CMD ["node", "index.js"]
--> Running in 1573fecbe305
--> Removed intermediate container 1573fecbe305
--> ddf7974f5bfc
Successfully built ddf7974f5bfc
Successfully tagged node-app:v1
root@ip-172-31-27-214:~/docker-tasks/NodeJs based_source# docker run -itd --name node_container -p 3000:3000 node-app:v1
255c35fdb3ccdc37bf280c9a8243964b07bee3bc7a1677f15d167d990b31025a
root@ip-172-31-27-214:~/docker-tasks/NodeJs based_source# docker tag node-app:v1 arjumand10/node-app:v1
root@ip-172-31-27-214:~/docker-tasks/NodeJs based_source# docker push arjumand10/node-app:v1
The push refers to repository [docker.io/arjumand10/node-app]
938bb6ef6936: Pushed
c638367c476c: Pushed [=====>] 112.2MB/378MB
881c8f67afb: Pushed
e28f53773a16: Pushed
82140d9a70a7: Mounted from library/node
f3b40b0cdbc1: Mounted from library/node
0a1f26057bd0: Mounted from library/node
08000c18d16d: Mounted from library/node
1
```

i-028f1dc9ea91e8b89 (Docker_controller)
PublicIPs: 98.81.232.104 PrivateIPs: 172.31.27.214

→Validation:



→ Validation on git hub repo



6. Write a docker-compose file to set up WordPress with a MySQL database.

Docker Compose (WordPress + MySQL)

Create file: docker-compose.yml

version: "3.8"

services:

mysql:

image: mysql:5.7

container_name: mysql_db

restart: always

environment:

MYSQL_ROOT_PASSWORD: root123

MYSQL_DATABASE: wordpress

MYSQL_USER: wpuser

MYSQL_PASSWORD: wp123

volumes:

- mysql_data:/var/lib/mysql

wordpress:

image: wordpress:latest

container_name: wordpress_app

restart: always

ports:

- "8080:80"

environment:

WORDPRESS_DB_HOST: mysql:3306

WORDPRESS_DB_USER: wpuser

WORDPRESS_DB_PASSWORD: wp123

WORDPRESS_DB_NAME: wordpress

depends_on:

- mysql


```
restart: always
environment:
  MYSQL_ROOT_PASSWORD: root123
  MYSQL_DATABASE: wordpress
  MYSQL_USER: wpuser
  MYSQL_PASSWORD: wp123
volumes:
  - mysql_data:/var/lib/mysql

wordpress:
  image: wordpress:latest
  container_name: wordpress_app
  restart: always
  ports:
    - "8080:80"
  environment:
    WORDPRESS_DB_HOST: mysql:3306
    WORDPRESS_DB_USER: wpuser
    WORDPRESS_DB_PASSWORD: wp123
    WORDPRESS_DB_NAME: wordpress
  depends_on:
    - mysql

volumes:
  mysql_data:
```

i-028f1dc9ea91e8b89 (Docker controller)

volumes:

mysql_data:

Run

docker-compose up -d

```
root@ip-172-31-27-214:~/docker-tasks# vi docker-compose.yml
root@ip-172-31-27-214:~/docker-tasks# apt install docker-compose
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker-compose is already the newest version (1.29.2-6ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 43 not upgraded.
root@ip-172-31-27-214:~/docker-tasks# docker-compose -version
docker-compose version 1.29.2, build unknown
root@ip-172-31-27-214:~/docker-tasks# docker-compose up -d
Creating network "docker-tasks_default" with the default driver
Creating volume "docker-tasks_mysql_data" with default driver
Pulling mysql (mysql:5.7)...
```

i-028f1dc9ea91e8b89 (Docker controller)

Access:

👉 <http://<server-ip>:8080>

